# Java Exception Handling Reference Guide

Comprehensive Practical Examples, Code Implementations & Test Case Verifications

**Subject:** Object-Oriented Programming via Java

**Topic:** Built-in Exceptions (Checked & Unchecked)

## 1. NullPointerException UNCHECKED EXCEPTION

### 1. Aim

To write and execute a Java program to demonstrate `NullPointerException` using a suitable example.

#### 2. DEFINITION

A `NullPointerException` occurs when a program tries to access or use an object reference that contains the value `null` (e.g., invoking a method, accessing a field, or getting the length of a null reference).

#### 3. STRUCTURE

```
try {
    // Code that may cause NullPointerException
} catch (NullPointerException e) {
    // Handling the exception
}
```

#### 4. PROGRAM

```
public class NullPointerExample {
    public static void main(String[] args) {
        String name = null;

        try {
            System.out.println("Length of name: " + name.length());
        } catch (NullPointerException e) {
            System.out.println("NullPointerException occurred!");
            System.out.println("String reference is null.");
        }
    }
}
```

#### 5. SAMPLE OUTPUT

```
NullPointerException occurred!
String reference is null.
```

### 6. TEST CASE VERIFICATION

| Test Case | Input | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | `name = null` | NullPointerException should be handled | NullPointerException occurred! | Pass |
| 2 | `name = "Dinkar"` | Length should be displayed | Length of name: 6 | Pass |

*Note for Test Case 2: Changing `String name = "Dinkar";` yields output: `Length of name: 6`.*

### 7. CONCLUSION

> *Thus, the Java program was successfully implemented to demonstrate `NullPointerException`. The exception was handled using the try-catch block, preventing the program from terminating abnormally.*

## 2. ArrayIndexOutOfBoundsException UNCHECKED EXCEPTION

### 1. AIM

To write and execute a Java program to demonstrate `ArrayIndexOutOfBoundsException` using a suitable example.

### 2. DEFINITION

An `ArrayIndexOutOfBoundsException` occurs when a program tries to access an array element using an index that is outside the valid range of the array (i.e., less than 0 or greater than or equal to array size).

### 3. STRUCTURE

```
try {
    // Code that may cause ArrayIndexOutOfBoundsException
} catch (ArrayIndexOutOfBoundsException e) {
    // Handling the exception
}
```

### 4. PROGRAM

```
public class ArrayIndexExample {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        try {
            System.out.println("Element at index 2: " + numbers[2]);
            System.out.println("Element at index 5: " + numbers[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException occurred!");
            System.out.println("Invalid array index.");
        }
    }
}
```

### 5. SAMPLE OUTPUT

```
Element at index 2: 30
ArrayIndexOutOfBoundsException occurred!
Invalid array index.
```

### 6. TEST CASE VERIFICATION

| Test Case | Input | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | `numbers[2]` | Element should be 30 | Element at index 2: 30 | Pass |
| 2 | `numbers[5]` | Exception should be handled | ArrayIndexOutOfBoundsException occurred! | Pass |
| 3 | `numbers[4]` | Element should be 50 | Element at index 4: 50 | Pass |

### 7. CONCLUSION

*Thus, the Java program was successfully implemented to demonstrate `ArrayIndexOutOfBoundsException`. The exception was successfully handled using the try-catch block when an invalid array index was accessed.*

## 3. NumberFormatException UNCHECKED EXCEPTION

### 1. AIM

To write and execute a Java program to demonstrate `NumberFormatException` using a suitable example.

### 2. DEFINITION

`NumberFormatException` occurs when a program tries to convert a String that does not contain a valid numeric format into a numeric type (such as integer or float).

### 3. STRUCTURE

```
try {
    // Code that may cause NumberFormatException
} catch (NumberFormatException e) {
    // Handling the exception
}
```

### 4. PROGRAM

```
public class NumberFormatExample {
    public static void main(String[] args) {
        String value = "ABC";

        try {
            int number = Integer.parseInt(value);
            System.out.println("Number: " + number);
        } catch (NumberFormatException e) {
            System.out.println("NumberFormatException occurred!");
            System.out.println("Invalid number format.");
        }
    }
}
```

### 5. SAMPLE OUTPUT

```
NumberFormatException occurred!
Invalid number format.
```

### 6. TEST CASE VERIFICATION

| Test Case | Input | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | `"ABC"` | Exception should be handled | NumberFormatException occurred! | Pass |
| 2 | `"123"` | Number should be displayed | Number: 123 | Pass |
| 3 | `"45A"` | Exception should be handled | NumberFormatException occurred! | Pass |

*Note for Test Case 2: Changing `String value = "123";` yields output: `Number: 123`.*

### 7. CONCLUSION

*Thus, the Java program was successfully implemented to demonstrate `NumberFormatException`. The exception was handled using the try-catch block when an invalid String was converted into an integer.*

## 4. StringIndexOutOfBoundsException UNCHECKED EXCEPTION

### 1. AIM

To write and execute a Java program to demonstrate `StringIndexOutOfBoundsException` using a suitable example.

### 2. DEFINITION

`StringIndexOutOfBoundsException` occurs when a program tries to access a character in a String using an index that is either negative or greater than or equal to the size of the String.

## 3. STRUCTURE

```
try {
    // Code that may cause StringIndexOutOfBoundsException
} catch (StringIndexOutOfBoundsException e) {
    // Handling the exception
}
```

## 4. PROGRAM

```
public class StringIndexExample {
    public static void main(String[] args) {
        String name = "Dinkar";

        try {
            System.out.println("Character at index 2: " + name.charAt(2));
            System.out.println("Character at index 10: " + name.charAt(10));
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("StringIndexOutOfBoundsException occurred!");
            System.out.println("Invalid String index.");
        }
    }
}
```

## 5. SAMPLE OUTPUT

```
Character at index 2: n
StringIndexOutOfBoundsException occurred!
Invalid String index.
```

## 6. TEST CASE VERIFICATION

| Test Case | Input | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | `name.charAt(2)` | Character n should be displayed | Character at index 2: n | Pass |
| 2 | `name.charAt(10)` | Exception should be handled | StringIndexOutOfBoundsException occurred! | Pass |
| 3 | `name.charAt(5)` | Character r should be displayed | Character at index 5: r | Pass |

**String Layout Analysis:**

The String `"Dinkar"` contains 6 characters with valid 0-based indexing:

| Character | D | i | n | k | a | r |
|---|---|---|---|---|---|---|
| Index | 0 | 1 | 2 | 3 | 4 | 5 |

Therefore, accessing index `10` exceeds the range [0..5] and throws `StringIndexOutOfBoundsException`.

### 7. CONCLUSION

*Thus, the Java program was successfully implemented to demonstrate `StringIndexOutOfBoundsException`. The exception was successfully handled using the try-catch block when an invalid String index was accessed.*

## 5. ClassCastException UNCHECKED EXCEPTION

### 1. AIM

To write and execute a Java program to demonstrate `ClassCastException` using a suitable example.

### 2. DEFINITION

`ClassCastException` occurs when code attempts to cast an object to a subclass of which it is not an instance (incompatible type casting).

### 3. STRUCTURE

```
try {
    // Code that may cause ClassCastException
} catch (ClassCastException e) {
    // Handling the exception
}
```

## 4. PROGRAM

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows");
    }
}

public class ClassCastExample {
    public static void main(String[] args) {
        Animal animal = new Dog();

        try {
            Cat cat = (Cat) animal;
            cat.meow();
        } catch (ClassCastException e) {
            System.out.println("ClassCastException occurred!");
            System.out.println("Object cannot be cast to Cat.");
        }
    }
}
```

## 5. SAMPLE OUTPUT

```
ClassCastException occurred!
Object cannot be cast to Cat.
```

## 6. EXPLANATION

1. The statement `Animal animal = new Dog();` assigns a `Dog` object to an `Animal` reference.
2. The explicit downcast `Cat cat = (Cat) animal;` attempts to treat the underlying `Dog` object as a `Cat`.
3. Since `Dog` and `Cat` are sibling classes under `Animal`, the JVM throws a `ClassCastException` at runtime.

### 7. TEST CASE VERIFICATION

| Test Case | Input / Statement | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | `Animal animal = new Dog();` cast to `Cat` | Exception should be handled | ClassCastException occurred! | Pass |
| 2 | `Animal animal = new Dog();` cast to `Dog` | Dog object accessed | No exception | Pass |
| 3 | `Animal animal = new Cat();` cast to `Cat` | Cat object accessed | No exception | Pass |

### 8. CONCLUSION

*Thus, the Java program was successfully implemented to demonstrate `ClassCastException`. The exception was successfully handled using the try-catch block when an object was incorrectly cast to an incompatible class.*

## 6. IOException CHECKED EXCEPTION

### 1. AIM

To write and execute a Java program to demonstrate `IOException` using a suitable example.

### 2. DEFINITION

`IOException` is a checked exception in Java that signals an I/O operation (reading, writing, file access, stream operations) has failed or been interrupted.

### 3. STRUCTURE

```
try {
    // Input/Output operation
} catch (IOException e) {
    // Handling the exception
}
```

## 4. PROGRAM

```java
import java.io.*;

public class IOExceptionExample {
    public static void main(String[] args) {
        try {
            FileReader file = new FileReader("sample.txt");
            int ch;

            while ((ch = file.read()) != -1) {
                System.out.print((char) ch);
            }

            file.close();
        } catch (IOException e) {
            System.out.println("IOException occurred!");
            System.out.println("Error while reading the file.");
        }
    }
}
```

## 5. SAMPLE OUTPUT

**If sample.txt does not exist:**

```
IOException occurred!
Error while reading the file.
```

**If sample.txt exists and contains "Hello Java":**

```
Hello Java
```

## 6. EXPLANATION

The statement `FileReader file = new FileReader("sample.txt");` attempts to open and read a file. If I/O failure occurs (file unreadable, missing, closed stream), Java throws an `IOException` which is caught and handled safely.

## 7. TEST CASE VERIFICATION

| Test Case | Condition | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | `sample.txt` exists | File contents displayed | File contents displayed | Pass |
| 2 | `sample.txt` missing | IOException handled | IOException occurred! | Pass |
| 3 | File unreadable | IOException handled | Error while reading file. | Pass |

## 8. CONCLUSION

*Thus, the Java program was successfully implemented to demonstrate `IOException`. The exception was handled using the try-catch block while performing a file input operation.*

# 7. FileNotFoundException CHECKED EXCEPTION

## 1. AIM

To write and execute a Java program to demonstrate `FileNotFoundException` using a suitable example.

## 2. DEFINITION

`FileNotFoundException` is a specific checked subclass of `IOException` that occurs when an attempt to open a file specified by a pathname fails because the file does not exist or is inaccessible.

## 3. STRUCTURE

```
try {
    // Code for opening a file
} catch (FileNotFoundException e) {
    // Handling the exception
}
```

## 4. PROGRAM

```
import java.io.*;

public class FileNotFoundExample {
    public static void main(String[] args) {
        try {
            FileReader file = new FileReader("student.txt");
            System.out.println("File opened successfully.");
            file.close();
        } catch (FileNotFoundException e) {
            System.out.println("FileNotFoundException occurred!");
            System.out.println("The file was not found.");
        } catch (IOException e) {
            System.out.println("IOException occurred!");
        }
    }
}
```

## 5. SAMPLE OUTPUT

**If student.txt does not exist:**

```
FileNotFoundException occurred!
The file was not found.
```

**If student.txt exists:**

```
File opened successfully.
```

## 6. EXPLANATION

`FileReader("student.txt")` looks for the file in the working path. If missing, `FileNotFoundException` triggers first before generic `IOException` due to catch-block hierarchy ordering.

### 7. TEST CASE VERIFICATION

| Test Case | Condition | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | `student.txt` missing | FileNotFoundException handled | FileNotFoundException occurred! | Pass |
| 2 | `student.txt` exists | File opened successfully | File opened successfully. | Pass |
| 3 | File restricted | FileNotFoundException handled | FileNotFoundException occurred! | Pass |

### 8. CONCLUSION

*Thus, the Java program was successfully implemented to demonstrate `FileNotFoundException`. The exception was successfully handled using the try-catch block when the specified file was unavailable.*

## 8. SQLException CHECKED EXCEPTION

### 1. AIM

To write and execute a Java program to demonstrate `SQLException` using a suitable example.

### 2. DEFINITION

`SQLException` is a checked exception in Java that provides information on database access errors, invalid SQL syntax, database connectivity issues, or missing tables/columns in JDBC.

### 3. STRUCTURE

```
try {
    // Database operation
} catch (SQLException e) {
    // Handling the exception
}
```

## 4. PROGRAM

```
import java.sql.*;

public class SQLExceptionExample {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/studentdb",
                "root",
                "password"
            );

            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM students");

            while (rs.next()) {
                System.out.println(
                    "ID: " + rs.getInt("id") +
                    ", Name: " + rs.getString("name")
                );
            }

            con.close();
        } catch (SQLException e) {
            System.out.println("SQLException occurred!");
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```

## 5. SAMPLE OUTPUT

**If DB connection or table is unavailable:**

```
SQLException occurred!
Database error: Table 'studentdb.students' doesn't exist
```

**If DB configured correctly:**

```
ID: 101, Name: Ravi
ID: 102, Name: Arun
```

## 6. EXPLANATION

When executing JDBC queries, any issue like failed connection credentials, bad table names, or invalid SQL queries generates an `SQLException` containing diagnostic error details.

### 7. TEST CASE VERIFICATION

| Test Case | Condition | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | DB connection correct | Student records displayed | Student records displayed | Pass |
| 2 | Table does not exist | SQLException handled | SQLException occurred! | Pass |
| 3 | Database name incorrect | SQLException handled | Database error displayed | Pass |
| 4 | SQL query invalid | SQLException handled | SQLException occurred! | Pass |

### 8. CONCLUSION

*Thus, the Java program was successfully implemented to demonstrate `SQLException`. The exception was handled using the try-catch block while performing database operations through JDBC.*

## 9. InterruptedException CHECKED EXCEPTION

### 1. AIM

To write and execute a Java program to demonstrate `InterruptedException` using a suitable example.

### 2. DEFINITION

`InterruptedException` is a checked exception thrown when a thread that is sleeping, waiting, or otherwise occupied is interrupted by another thread using `interrupt()`.

### 3. STRUCTURE

```
try {
    Thread.sleep(5000);
} catch (InterruptedException e) {
    // Handling the exception
}
```

## 4. PROGRAM

```java
public class InterruptedExceptionExample {
    public static void main(String[] args) {
        Thread t = new Thread(() -> {
            try {
                System.out.println("Thread is sleeping...");
                Thread.sleep(5000);
                System.out.println("Thread completed.");
            } catch (InterruptedException e) {
                System.out.println("InterruptedException occurred!");
                System.out.println("Thread was interrupted.");
            }
        });

        t.start();

        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }

        t.interrupt();
    }
}
```

## 5. SAMPLE OUTPUT

```
Thread is sleeping...
InterruptedException occurred!
Thread was interrupted.
```

## 6. EXPLANATION

The child thread starts sleeping for 5 seconds. The main thread sleeps for 1 second and then calls `t.interrupt()`. This interrupts the sleeping child thread immediately, triggering the `InterruptedException` block.

## 7. TEST CASE VERIFICATION

| Test Case | Condition | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | Interrupted during sleep() | InterruptedException handled | InterruptedException occurred! | Pass |
| 2 | No interruption | Thread completed | Thread completed. | Pass |
| 3 | Interrupted while waiting | Exception handled | Thread was interrupted. | Pass |

## 8. CONCLUSION

*Thus, the Java program was successfully implemented to demonstrate `InterruptedException`. The exception was successfully handled when a sleeping thread was interrupted using the interrupt() method.*

# 10. ClassNotFoundException CHECKED EXCEPTION

## 1. AIM

To write and execute a Java program to demonstrate `ClassNotFoundException` using a suitable example.

## 2. DEFINITION

`ClassNotFoundException` is a checked exception thrown when an application tries to load in a class through its string name using methods like `Class.forName()`, `ClassLoader.loadClass()`, but no definition for the class could be found on the classpath.

## 3. STRUCTURE

```
try {
    Class.forName("ClassName");
} catch (ClassNotFoundException e) {
    // Handling the exception
}
```

## 4. PROGRAM

```
public class ClassNotFoundExample {
    public static void main(String[] args) {
        try {
            Class.forName("Student");
            System.out.println("Class found successfully.");
        } catch (ClassNotFoundException e) {
            System.out.println("ClassNotFoundException occurred!");
            System.out.println("The specified class was not found.");
        }
    }
}
```

## 5. SAMPLE OUTPUT

```
ClassNotFoundException occurred!
The specified class was not found.
```

## 6. EXPLANATION

The method `Class.forName("Student")` attempts dynamically to locate and load the bytecodes for class `Student` at runtime. Since no such class exists in the classpath, the runtime throws `ClassNotFoundException`.

## 7. TEST CASE VERIFICATION

| Test Case | Input / Condition | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| 1 | `Class.forName("Student")` (unavailable) | Exception handled | ClassNotFoundException occurred! | **Pass** |
| 2 | `Class.forName("java.lang.String")` | Class found | Class found successfully. | **Pass** |
| 3 | Invalid class name | Exception handled | ClassNotFoundException occurred! | **Pass** |

*Note for Test Case 2: Changing `Class.forName("java.lang.String");` yields output: `Class found successfully.`.*

## 8. CONCLUSION

*Thus, the Java program was successfully implemented to demonstrate `ClassNotFoundException`. The exception was successfully handled using the try-catch block when the specified class could not be found at runtime.*